HTTP Methods and Status Codes

1. Introduction:

- HTTP methods and status codes are **core components** of the client–server communication process.
- Methods define **what action** a client wants to perform on a resource.
- Status codes indicate **the result** of the server's attempt to process that action.
- Together, they make HTTP a **robust and stateless** protocol.

PART 1: HTTP METHODS

2. Overview of HTTP Methods:

- HTTP defines several **request methods** that specify the desired action to be performed on a resource.
- Each method has a distinct purpose and expected behavior.
- Methods can be **safe**, **idempotent**, or **cacheable**:
  - **Safe:** Does not modify the server state (e.g., GET, HEAD).
  - **Idempotent:** Produces the same result if executed multiple times (e.g., PUT, DELETE).
  - **Cacheable:** Response can be stored for reuse (e.g., GET, HEAD).

Method	Description	Safe	Idempotent	Cacheable
GET	Requests data from the server.	Yes	Yes	Yes
POST	Sends data to the server to create a resource.	No	No	Sometimes
PUT	Replaces an existing resource with new data.	No	Yes	No
DELETE	Removes a resource from the server.	No	Yes	No
HEAD	Retrieves only headers, not body.	Yes	Yes	Yes
OPTIONS	Returns supported methods for a resource.	Yes	Yes	No
PATCH	Applies partial modification to a resource.	No	No	No

3. Common HTTP Methods:4. Example of GET Request:

```
GET /index.html HTTP/1.1
Host: www.example.com
Accept: text/html
Connection: keep-alive
```

Explanation:

- The browser asks the server for “index.html”.
- No data is sent in the body.
- The server returns the requested webpage.

4. Example of GET Request:

```
GET /index.html HTTP/1.1
Host: www.example.com
Accept: text/html
Connection: keep-alive
```

Explanation:

- The browser asks the server for “index.html”.
- No data is sent in the body.
- The server returns the requested webpage.

5. Example of POST Request:

```
POST /login HTTP/1.1
 Host: www.example.com
 Content-Type: application/x-www-form-urlencoded
 Content-Length: 32

username=Aayush&password=12345
```

Explanation:

- POST sends data to the server to process.
- Commonly used for forms and authentication.

6. Example of PUT Request:

```
PUT /users/123 HTTP/1.1
 Host: api.example.com
 Content-Type: application/json
```

```
{
 "name": "Aayush",
 "email": "aayush@example.com"
          }
```

Explanation:

- PUT replaces the existing resource with new data.
- Used in REST APIs to update records.

7. Example of DELETE Request:

```
DELETE /users/123 HTTP/1.1
Host: api.example.com
```

Explanation:

- The client asks the server to delete the resource at /users/123.
- If successful, the server responds with a 200 OK or 204 No Content.

8. Example of HEAD Request:

```
HEAD /index.html HTTP/1.1
 Host: www.example.com
```

Explanation:

- Similar to GET, but retrieves only headers.
- Useful for checking resource availability or metadata.

9. Example of OPTIONS Request:

```
OPTIONS /api/users HTTP/1.1
 Host: api.example.com
```

Explanation:

- Used by browsers and APIs to determine which HTTP methods are supported.
- Commonly used in **CORS (Cross-Origin Resource Sharing)** preflight checks.

10. Example of PATCH Request:

```
PATCH /users/123 HTTP/1.1
 Host: api.example.com
 Content-Type: application/json
```

```
{
 "email": "newmail@example.com"
        }
```

Explanation:

- Updates only part of the resource.
- More efficient than PUT when modifying small portions.

PART 2: HTTP STATUS CODES

11. Introduction to Status Codes:

- HTTP status codes are **3-digit numbers** sent by the server to indicate the result of a request.
- They help clients understand if a request succeeded, failed, or needs further action.
- The first digit defines the **response category**:
  - 1xx – Informational
  - 2xx – Success
  - 3xx – Redirection
  - 4xx – Client Error
  - 5xx – Server Error

12. 1xx – Informational Responses:

Code	Message	Description
100	Continue	Request part received, continue sending.
101	Switching Protocols	Server switching to another protocol (e.g., WebSocket).

13. 2xx – Successful Responses:

Code	Message	Description
200	OK	Request processed successfully.
201	Created	Resources successfully created (e.g., POST success).
202	Accepted	Request accepted for processing but not completed.
204	No Content	Request successful but no data to return.
206	Partial Content	Partial response for range requests.

Example:
HTTP/1.1 200 OK
Content-Type: text/html

<html> <body>Success!</body> </html>

14. 3xx – Redirection Responses:

Code	Message	Description
301	Moved Permanently	Resource moved to a new URL.
302	Found	Temporary redirection.
304	Not Modified	Cached version can be used.
307	Temporary Redirect	Same as 302 but maintains request method.
308	Permanent Redirect	Similar to 301 but maintains method.

Example:
HTTP/1.1 301 Moved Permanently
Location: https://www.newsite.com/

15. 4xx – Client Error Responses:

Code	Message	Description
400	Bad Request	Invalid syntax or malformed request.
401	Unauthorized	Authentication required.
403	Forbidden	Client not allowed to access the resource.
404	Not Found	Requested resource not available.
405	Method Not Allowed	Request method not supported by the resource.
408	Request Timeout	Client took too long to send request.
429	Too Many Requests	Client sent too many requests in a short period.

Example:
HTTP/1.1 404 Not Found
Content-Type: text/html

<html> <body>Resource not found!</body> </html>

16. 5xx – Server Error Responses:

Code	Message	Description
500	Internal Server Error	Generic server failure.
501	Not Implemented	Request method not supported by the server.
502	Bad Gateway	Invalid response from upstream server.
503	Service Unavailable	Server temporarily overloaded or down.
504	Gateway Timeout	Server did not receive timely response.
505	HTTP Version Not Supported	Server doesn't support the requested HTTP version.

Example:
HTTP/1.1 503 Service Unavailable
Retry-After: 120

<html> <body>Server is temporarily unavailable.</body> </html>

17. Real-World Use Cases:

- **200 OK:** Loading a webpage successfully.
- **201 Created:** Uploading a file or creating a new record in an API.
- **301 Moved Permanently:** Redirecting users from old domain to new one.
- **404 Not Found:** Missing or deleted web pages.
- **500 Internal Server Error:** Unhandled exceptions in server-side code.

18. Practical Debugging Example:

Scenario: Submitting a login form.

1. Client sends a POST request to /login.
2. Server validates credentials.
3. If successful → 200 OK or 302 Redirect (to dashboard).
4. If invalid → 401 Unauthorized.
5. If server crashes → 500 Internal Server Error.

This helps developers identify where a request fails.

Summary

- HTTP methods define **what action** the client performs on a resource.
- HTTP status codes define **how the server responds** to those actions.
- Understanding both helps in **debugging, designing APIs, and web development.**
- Proper use ensures efficient communication between clients and servers.